

• Minimax 게임 설계

Minimax 알고리즘을 실제 코드로 구현하기 위해서는 먼저 알고리즘이 동작할 수 있는 간단한 게임 환경을 설계해야 한다. 체스는 말의 종류, 이동 규칙, 체크메이트 판정, 특수 규칙 등이 복잡하기 때문에 처음부터 Minimax를 적용하기에는 구현 난도가 높다. 따라서 이번 절에서는 3×3 보드에서 진행되는 틱택토 게임을 예제로 사용한다.

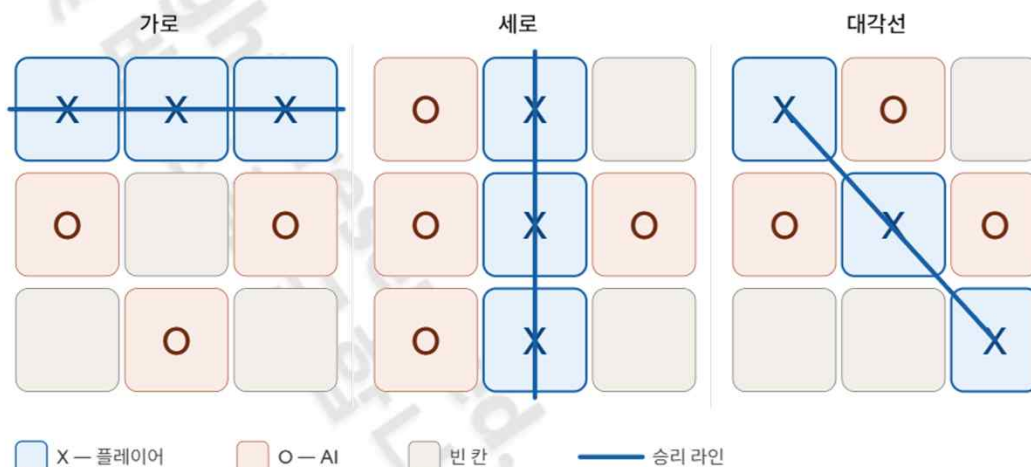
틱택토는 두 플레이어가 번갈아 가며 빈 칸에 자신의 기호를 놓고, 가로·세로·대각선 중 한 줄을 먼저 완성하면 승리하는 게임이다. 규칙은 단순하지만, 가능한 수를 생성하고 미래 상태를 탐색하며 최적의 수를 선택해야 한다는 점에서 Minimax 알고리즘의 핵심 구조를 학습하기에 적합하다.

1. 틱택토 게임의 기본 규칙

이번 예제에서는 사람 플레이어를 X, 인공지능을 O로 설정한다. 사람은 먼저 빈 칸을 선택해 X를 놓고, 이후 AI가 Minimax 알고리즘을 이용해 가장 좋은 위치에 O를 놓는다. 이 과정은 한쪽이 승리하거나 더 이상 둘 수 있는 칸이 없어 무승부가 될 때까지 반복된다.



틱택토에서 승리 조건은 다음과 같다.



Minimax 알고리즘 구현

2. 보드 구조 설계

틱택토 보드는 3×3 형태이지만, 코드에서는 1차원 리스트로 표현한다. 2차원 배열을 사용해도 되지만, 1차원 리스트를 사용하면 각 칸을 0번부터 8번까지의 번호로 쉽게 관리할 수 있다.

```
board = [" "] * 9
```

각 번호는 다음과 같은 위치를 의미한다.

```
0 | 1 | 2
--+-+--
3 | 4 | 5
--+-+--
6 | 7 | 8
```

예를 들어 사용자가 4를 입력하면 가운데 칸에 X가 놓인다.

```
  |  |
--+-+--
  | x |
--+-+--
  |  |
```

3. 상태(State) 표현

Minimax 알고리즘에서 상태(State)는 현재 게임이 어떤 상황에 있는지를 나타내는 정보이다. 틱택토에서는 보드에 놓인 X, O, 빈 칸의 배치가 하나의 상태가 된다.

예를 들어 다음과 같은 보드가 있다고 하자.

```
x |  | o
--+-+--
  | x |
--+-+--
  |  | o
```

Minimax 알고리즘 구현

이 상태는 코드에서 다음과 같이 표현할 수 있다.

```
board = [  
    "X", " ", "O",  
    " ", "X", " ",  
    " ", " ", "O"  
]
```

즉, 틱택토의 상태는 현재 보드 리스트 하나로 표현할 수 있으며, Minimax 알고리즘은 이 상태를 기준으로 가능한 다음 수를 탐색한다.

4. 가능한 수 설계

틱택토에서 가능한 수는 현재 비어 있는 칸이다. 따라서 보드 리스트에서 값이 " "인 칸의 인덱스를 찾으면 현재 둘 수 있는 모든 수를 구할 수 있다.

```
def legal_moves(b):  
    return [i for i in range(9) if b[i] == " "]
```

예를 들어 현재 보드가 다음과 같다면,

```
x |   | o  
---+---  
  | x |  
---+---  
  |   | o
```

빈 칸은 1, 3, 5, 6, 7이므로 가능한 수는 다음과 같다.

```
[1, 3, 5, 6, 7]
```

이 함수는 이후 Minimax가 각 후보 수를 하나씩 가정해 보고, 그 결과를 재귀적으로 탐색하는 데 사용된다.

승패 판정과 평가 함수 설계

• 승패 판정과 평가 함수 설계

앞에서 틱택토 보드와 가능한 수를 설계하였다. 그러나 Minimax 알고리즘이 올바르게 동작하기 위해서는 현재 상태가 승리, 패배, 무승부 중 어느 상황인지 판단할 수 있어야 한다. 또한 게임이 종료된 상태에 도달했을 때, 그 결과를 수치화하여 평가값으로 반환해야 한다. 이번 절에서는 틱택토의 승리 조건을 코드로 표현하고, AI가 사용할 평가 함수를 설계한다.

1. 승리 조건 설계

틱택토에서 승리하려면 같은 기호가 가로, 세로, 대각선 중 하나의 줄에 연속으로 3개 놓여야 한다. 3×3 보드는 0번부터 8번까지의 인덱스로 표현하므로, 승리 가능한 조합을 미리 정리할 수 있다.

```
0 | 1 | 2
---+---+---
3 | 4 | 5
---+---+---
6 | 7 | 8
```

틱택토의 승리 조합은 다음과 같다.

```
win_cases = [
    (0, 1, 2), (3, 4, 5), (6, 7, 8), # 가로
    (0, 3, 6), (1, 4, 7), (2, 5, 8), # 세로
    (0, 4, 8), (2, 4, 6)             # 대각선
]
```

이 조합 중 하나라도 같은 기호로 채워져 있다면 해당 기호가 승자가 된다. 예를 들어 (0, 4, 8) 위치가 모두 X라면 플레이어가 대각선으로 승리한 것이다.

```
x |   | o
---+---+---
  | x |
---+---+---
o |   | x
```

승패 판정과 평가 함수 설계

2. 게임 종료 조건

Minimax 알고리즘은 게임이 끝난 상태에 도달하면 더 이상 탐색하지 않고 결과를 반환해야 한다. 따라서 다음 세 가지 경우를 확인해야 한다.

- ① X가 승리한 경우
- ② O가 승리한 경우
- ③ 빈 칸이 없어 무승부가 된 경우

이를 코드로 작성하면 다음과 같다.

```
def winner(b):
    win_cases = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8),
        (0, 3, 6), (1, 4, 7), (2, 5, 8),
        (0, 4, 8), (2, 4, 6)
    ]

    for a, c, d in win_cases:
        if b[a] == b[c] == b[d] and b[a] != " ":
            return b[a]

    if " " not in b:
        return "DRAW"

    return None
```

이 함수는 현재 보드 상태를 입력받아 다음 중 하나를 반환한다.

"X" → 플레이어 승리

"O" → AI 승리

"DRAW" → 무승부

None → 아직 게임 진행 중

예를 들어 다음 보드는 O가 세로로 승리한 상태이다.

```
x | o | x
---+---+---
  | o |
---+---+---
x | o |
```

승패 판정과 평가 함수 설계

이 경우 `winner(board)` 함수는 "O"를 반환한다.

3. 평가값 설계

Minimax 알고리즘은 게임 결과를 단순한 문자열이 아니라 점수로 비교해야 한다. 따라서 승리, 패배, 무승부를 수치로 변환하는 평가값이 필요하다. 이번 예제에서는 AI를 O, 사람 플레이어를 X로 설정한다. AI는 자신의 승리를 가장 좋은 결과로 판단해야 하므로 +10, 사람이 이긴 경우는 AI에게 불리하므로 -10, 무승부는 0으로 설정한다.

AI(O) 승리 : +10

플레이어(X) 승리 : -10

무승부 : 0

이 평가값은 Minimax 함수의 종료 조건에서 사용된다.

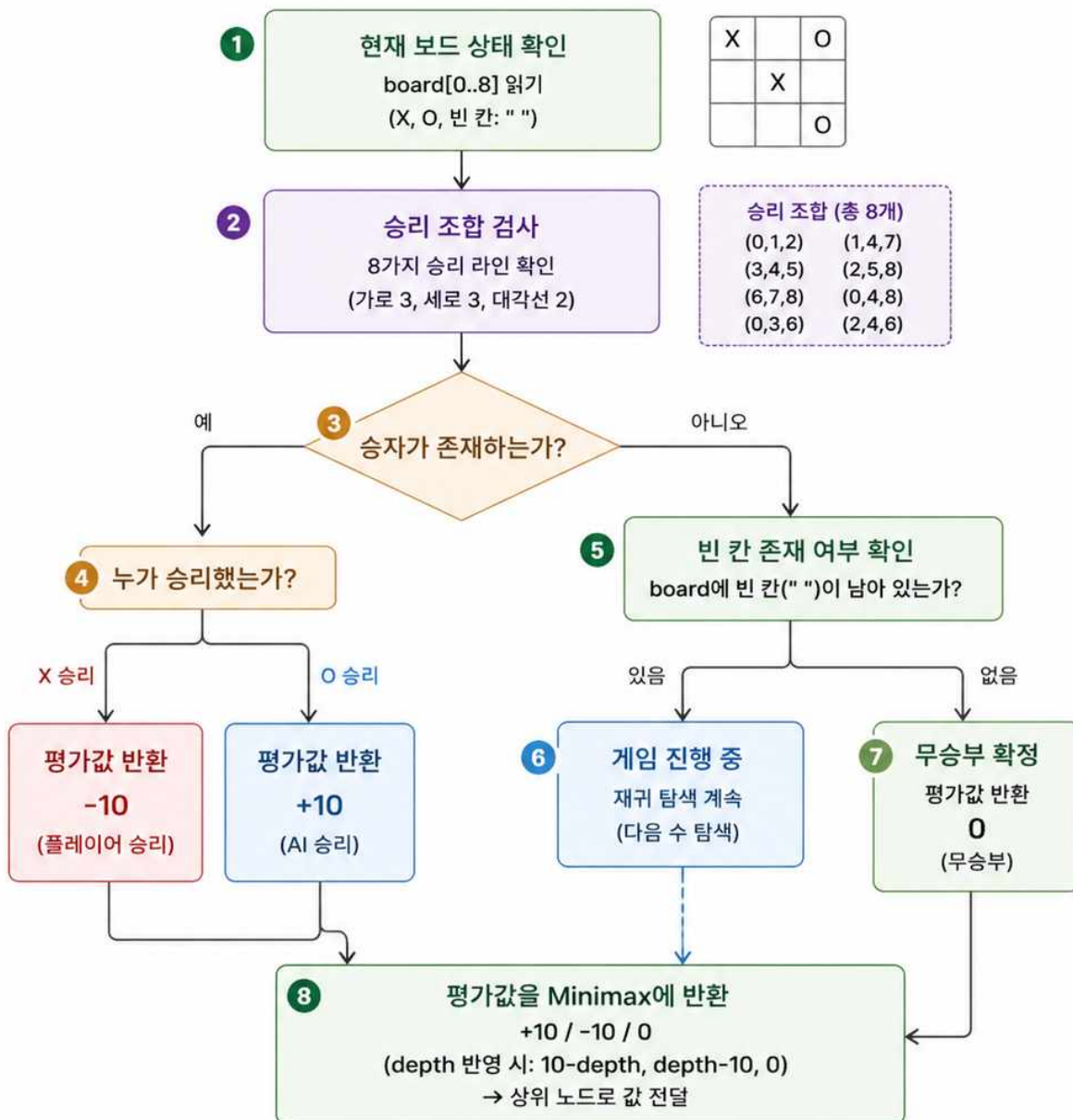
```
result = winner(b)

if result == "O":
    return 10
elif result == "X":
    return -10
elif result == "DRAW":
    return 0
```

즉, Minimax는 말단 상태에 도달했을 때 이 평가값을 상위 노드로 반환한다. 이후 AI 차례에서는 가장 큰 값을 선택하고, 사람 차례에서는 가장 작은 값을 선택하게 된다

승패 판정과 평가 함수는 Minimax 알고리즘의 기저 조건을 구성한다. 재귀 탐색이 계속 진행되다가 승리, 패배, 무승부 상태에 도달하면 더 이상 탐색하지 않고 평가값을 반환한다. 그림은 Minimax 알고리즘에서 현재 보드 상태를 검사하고 평가값을 반환하는 과정을 보여준다. 먼저 현재 보드 상태를 읽고 8개의 승리 조합(가로·세로·대각선)을 검사한다. 승자가 존재하면 누가 승리했는지를 판별하여 평가값을 반환한다. AI(O)가 승리하면 +10, 플레이어(X)가 승리하면 -10의 점수를 부여한다. 승자가 없다면 빈 칸이 남아 있는지 확인한다. 빈 칸이 없다면 게임은 무승부이므로 0점을 반환한다. 반대로 빈 칸이 남아 있다면 게임은 아직 진행 중이므로 재귀 탐색을 계속 수행하며 다음 상태를 탐색한다. 이렇게 계산된 평가값은 Minimax 알고리즘의 반환값으로 상위 노드로 전달되며, 이후 Max는 최대값, Min은 최소값을 선택하여 최적의 수를 결정하게 된다

승패 판정과 평가 함수 설계



정리하면, Minimax 알고리즘의 승패 판정과 평가 함수는 먼저 현재 보드 상태를 확인하고, 이를 바탕으로 승리, 무승부 또는 게임 진행 중인 상태인지 판별하는 과정으로 시작된다. 이후 승패가 결정된 상태라면 그 결과를 평가값(+10, 0, -10)으로 변환하여 반환하고, 이 값은 다시 상위 노드로 전달되어 Minimax 탐색 과정의 의사결정 기준으로 활용된다. 즉, 승패 판정과 평가 함수는 재귀 탐색을 종료시키는 기저 조건(Base Case) 이면서 동시에 각 상태의 유리함을 수치화하여 최적 수 선택을 가능하게 하는 핵심 기준 역할을 수행한다.

재귀 함수로 Minimax 구현

앞 절에서는 게임 상태를 평가하는 승패 판정과 평가 함수를 설계하였다. 이제 이를 바탕으로 Minimax 알고리즘의 핵심인 재귀 탐색 함수를 구현한다. Minimax는 단순히 현재 가장 좋아 보이는 수를 선택하는 방식이 아니라, 가능한 미래 상태를 재귀적으로 탐색하고 그 결과를 역으로 전달받아 현재 최적의 수를 결정하는 알고리즘이다.

이 과정의 핵심은 재귀 호출(Recursive Call) 과 결과값 반환(Return) 구조에 있다. 탐색은 하위 노드로 내려가며 진행되고, 계산된 평가값은 다시 상위 노드로 전달되면서 최적 수가 결정된다

1. 재귀 함수의 기본 구조

Minimax 함수는 일반적으로 다음 세 가지 입력을 가진다.

```
minimax(state, depth, is_maximizing)
```

각 인자의 의미는 다음과 같다.

- state : 현재 게임 상태(보드)
- depth(현재 탐색 깊이) : 얼마나 깊이 탐색했는지 나타낸다.
- is_maximizing(Max Player 여부) : 현재 차례가 AI(Max)인지 상대(Min)인지 구분한다.

기본 구조는 다음과 같다.

```
def minimax(board, is_maximizing):  
    result = winner(board)  
    if result is not None:  
        return evaluate(result)  
    if is_maximizing:  
        best_score = -999  
        for move in legal_moves(board):  
            board[move] = "O"  
            score = minimax(  
                board,  
                False  
            )  
            board[move] = ""  
            best_score = max(  
                best_score,  
                score
```

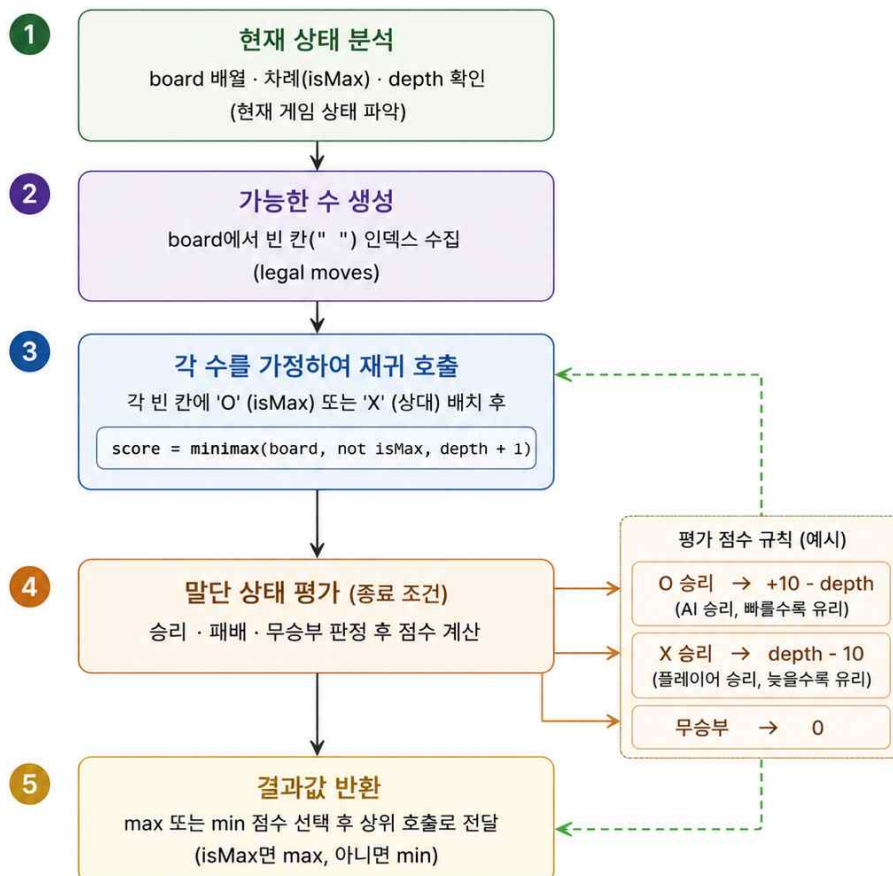

재귀 함수로 Minimax 구현

```

    )
    return best_score
else:
    best_score = 999
    for move in legal_moves(board):
        board[move] = "X"
        score = minimax(
            board,
            True
        )
        board[move] = " "
        best_score = min(
            best_score,
            score
        )
    return best_score

```

이 구조는 다음과 같은 재귀 탐색 흐름으로 동작한다.

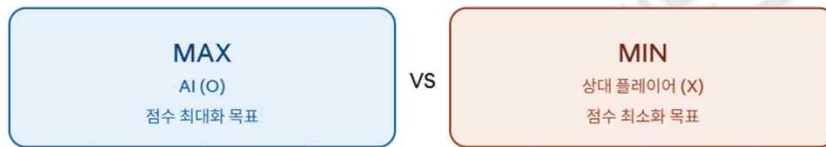


즉, Minimax는 깊이 우선 탐색(DFS) 구조와 동일한 형태로 작동한다

재귀 함수로 Minimax 구현

2. Max Player와 Min Player 구현

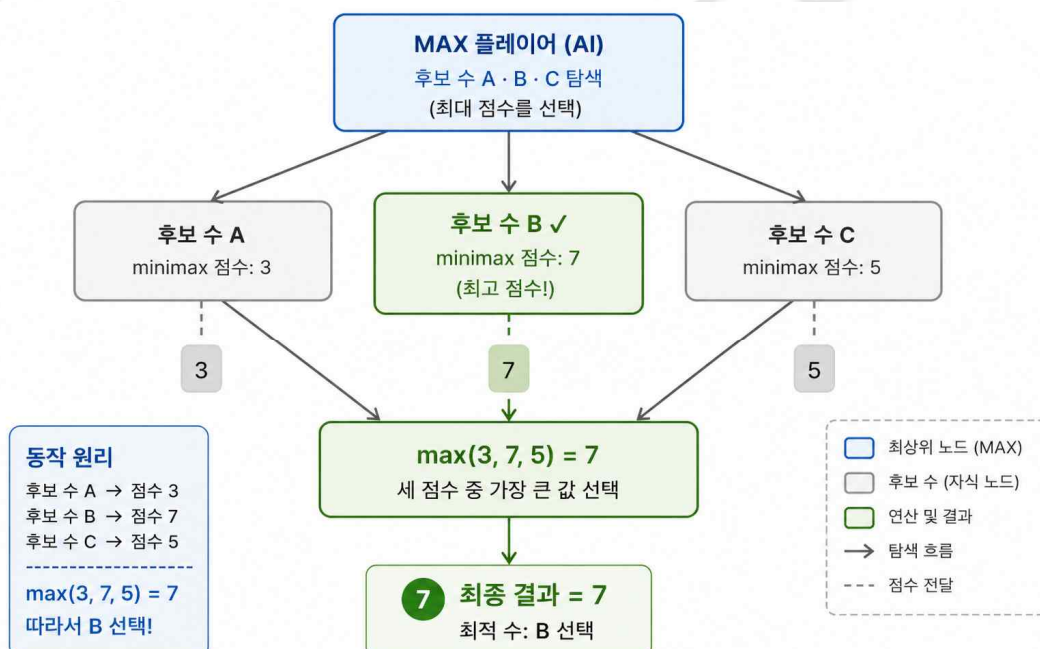
Minimax의 핵심은 두 플레이어가 서로 반대 전략을 가진다는 점이다.



(1) Max Player 구현

AI 차례에서는 가능한 수들 가운데 가장 큰 평가값을 선택해야 한다

```
best_score = -999
for move in legal_moves(board):
    board[move] = "O"
    score = minimax(
        board,
        False
    )
    board[move] = ""
    best_score = max(
        best_score,
        score
    )
return best_score
```



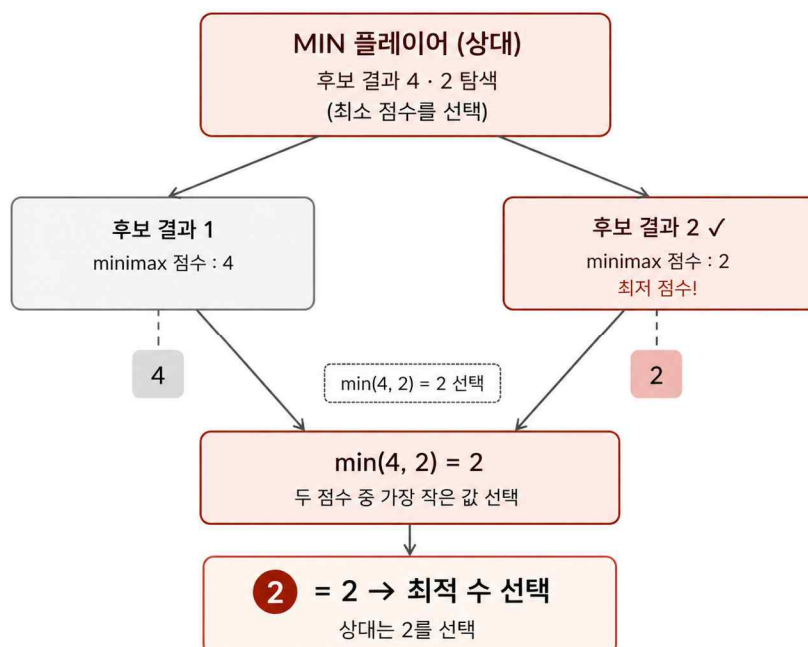
재귀 함수로 Minimax 구현

동작 원리를 살펴보면, Max Player인 AI는 가능한 후보 수 각각에 대해 Minimax 탐색을 수행하여 평가 점수를 계산한다. 예를 들어 후보 수 A의 평가값이 3, 후보 수 B의 평가값이 7, 후보 수 C의 평가값이 5라고 하면, AI는 이들 중 가장 큰 값을 선택해야 한다. 따라서 최대값 연산인 $\max(3, 7, 5) = 7$ 이 계산되며, 가장 높은 평가값을 가지는 후보 수 B가 최적의 선택으로 결정된다. 즉, Max Player는 가능한 수들 가운데 가장 높은 평가값을 보장하는 수를 선택함으로써 자신의 이익을 최대화한다.

(2) Min Player 구현

상대 차례에서는 AI에게 가장 불리한 결과를 만들도록 최소값을 선택한다.

```
best_score=999
for move in legal_moves(board):
    board[move]="X"
    score=minimax(
        board,
        True
    )
    board[move]=" "
    best_score=min(
        best_score,
        score
    )
return best_score
```



재귀 함수로 Minimax 구현

예를 들어 상대(Min Player)의 차례에서 가능한 후보 결과가 4와 2라고 가정하자. Min Player는 자신의 원리에 따라 가능한 결과들 중 가장 작은 값을 선택하므로 $\min(4, 2) = 2$ 가 계산된다. 따라서 상대는 평가값이 더 작은 2를 선택하게 된다. 이는 상대가 항상 AI에게 가장 불리한 결과를 유도한다고 가정하는 Minimax 알고리즘의 최소화(Minimize) 원리를 보여주는 예이다. 즉, Min Player는 여러 후보 중 최솟값을 선택하여 상대의 이익을 최소화하도록 동작한다.

(3) 종료 조건과 반환값

재귀 함수가 끝없이 호출되지 않기 위해서는 종료 조건(Base Case)이 반드시 필요하다.

Minimax 종료 조건은 다음과 같다.



이를 코드로 작성하면 다음과 같다.

```
result = winner(board)
if result == "O":
    return 10
elif result == "X":
    return -10
elif result == "DRAW":
    return 0
```

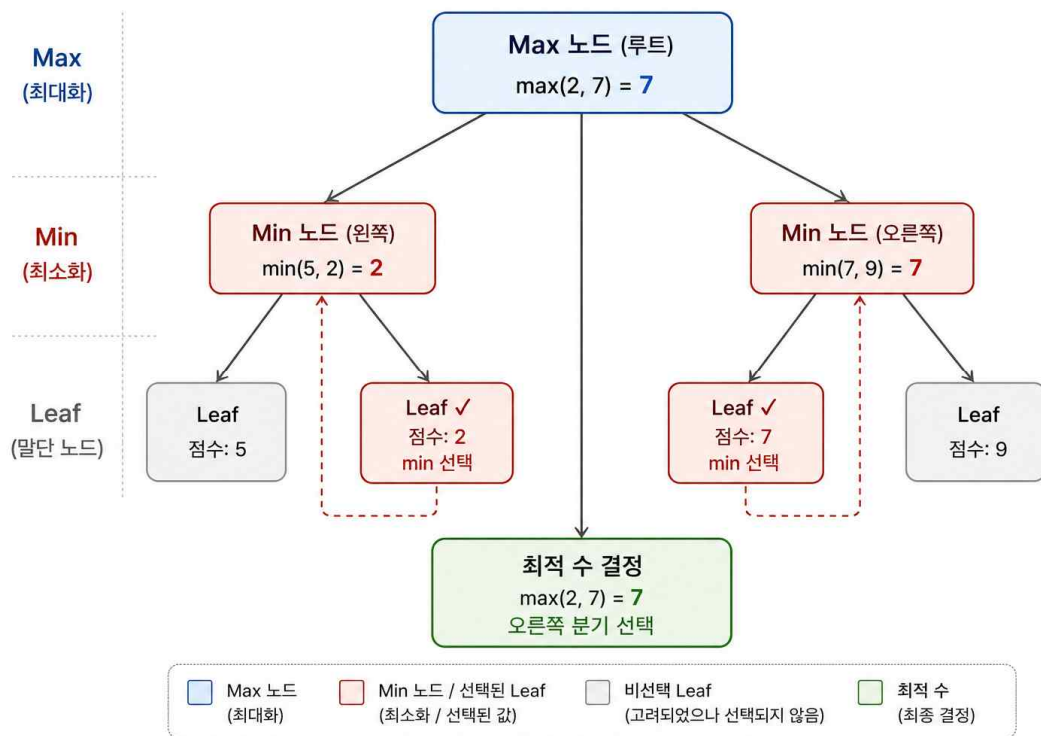
평가값의 의미는 현재 게임 상태가 AI에게 얼마나 유리하거나 불리한지를 수치로 나타내는 기준이다. AI가 승리한 상태는 가장 바람직한 결과이므로 +10의 높은 점수를 부여하고, 무승부는 어느 한쪽에도 유리하지 않은 중립 상태이므로 0으로 평가한다.

재귀 함수로 Minimax 구현

반면 플레이어가 승리한 경우는 AI에게 가장 불리한 결과이므로 -10의 점수를 부여한다. 이러한 평가값을 통해 Minimax 알고리즘은 가능한 미래 상태들을 비교하고, AI에게 가장 유리한 방향으로 최적의 수를 선택하게 된다.

3. 반환값 역전파

Minimax 알고리즘에서 말단 노드(Leaf Node)에서 계산된 평가값은 단순히 그 지점에서 끝나는 것이 아니라, 재귀 호출이 종료되면서 상위 노드로 다시 전달(역전파)된다. 이 과정에서 Min Player 노드는 전달받은 값들 중 최소값(min)을 선택하고, Max Player 노드는 최대값(max)을 선택한다. 이러한 선택 과정이 하위 노드에서 루트 노드까지 반복되면서 최종적으로 현재 상태에서 선택해야 할 최적의 수가 결정된다.



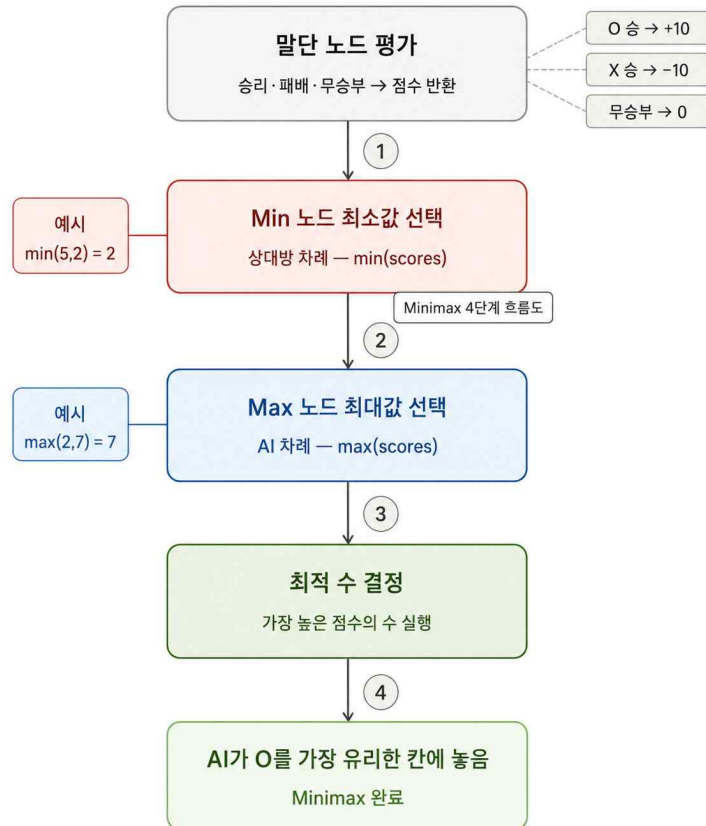
예를 들어 다음과 같은 평가값이 계산되었다고 하자. 왼쪽 Min 노드는 말단 노드의 평가값 5와 2 중 최소값을 선택하므로 $\min(5, 2) = 2$ 를 반환한다. 마찬가지로 오른쪽 Min 노드는 평가값 7과 9 중 최소값을 선택하여 $\min(7, 9) = 7$ 을 반환한다, 이렇게 계산된 두 값 2와 7은 상위 루트 노드(Max 노드)로 전달되며, Max 노드는 두 값 중 최대값을 선택한다.

$$\max(2, 7) = 7$$

최적 수 선택 과정 구현

따라서 평가값이 더 큰 오른쪽 분기(7)가 최적의 선택이 된다

이 과정은 다음과 같이 정리할 수 있다.



즉, 반환값 역전파는 단순히 점수를 위로 전달하는 과정이 아니라, 하위 탐색 결과를 바탕으로 현재 최적의 수를 결정하게 만드는 Minimax의 핵심 메커니즘이라 할 수 있다

3. 최적 수 선택 과정 구현

앞 절에서는 Minimax 재귀 함수가 가능한 미래 상태를 탐색하고, 말단 노드의 평가값을 상위 노드로 전달하는 과정을 살펴보았다. 이제 이 평가값을 실제로 활용하여 AI가 현재 보드에서 어떤 칸에 O를 놓아야 하는지 결정하는 과정을 구현한다.

최적 수 선택 과정은 다음 세 단계로 이루어진다.



최적 수 선택 과정 구현

(1) 가능한 수 하나씩 가정하기

AI가 수를 선택하려면 먼저 현재 보드에서 둘 수 있는 모든 빈 칸을 확인해야 한다. 틱택토에서는 빈 칸이 곧 가능한 수가 되므로, `legal_moves()` 함수를 이용해 후보 위치를 가져온다.

```
for move in legal_moves(board):
```

이 반복문은 현재 비어 있는 칸을 하나씩 꺼내면서, AI가 그 칸에 O를 놓았다고 가정한다.

```
board[move] = AI
```

이때 실제 게임을 바로 확정하는 것이 아니라, “만약 AI가 이 칸에 둔다면 어떤 결과가 나올까?”를 미리 실험하는 것이다. 따라서 점수 계산이 끝난 뒤에는 반드시 해당 칸을 다시 빈 칸으로 되돌려야 한다.

```
board[move] = EMPTY
```

이 과정을 백트래킹(Backtracking)이라고 한다.

(2) Minimax 점수 계산

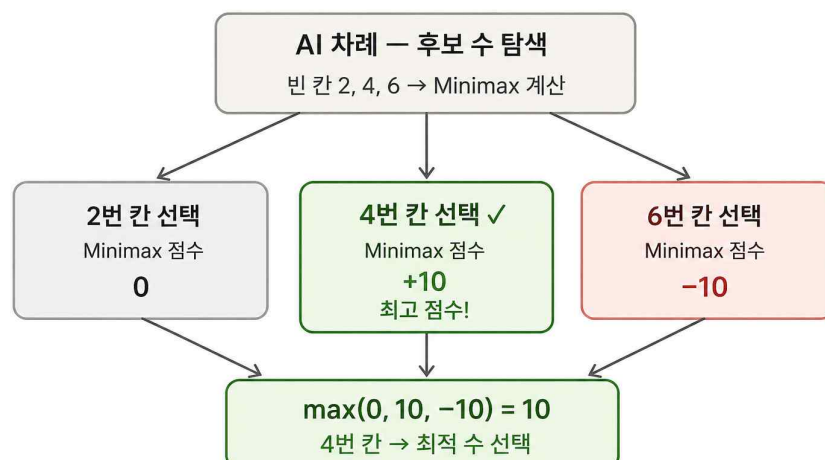
AI가 특정 칸에 O를 놓았다고 가정한 뒤에는, 그 다음 차례인 플레이어가 최선의 대응을 한다고 보고 Minimax 탐색을 수행한다.

```
score = minimax(board, False)
```

여기서 `False`는 다음 차례가 AI가 아니라 상대, 즉 Min Player의 차례임을 의미한다.

Minimax 함수는 해당 수 이후의 가능한 모든 미래 상태를 탐색하고, 최종적으로 그 수가 AI에게 얼마나 유리한지를 점수로 반환한다.

예를 들어 현재 가능한 수가 2, 4, 6번 칸이라고 하자. 각각의 후보 수에 대해 Minimax가 다음과 같은 점수를 반환할 수 있다.



최적 수 선택 과정 구현

이 경우 4번 칸은 AI가 승리할 가능성이 가장 높은 수이고, 6번 칸은 AI에게 불리한 수로 해석할 수 있다.

(3) 가장 높은 점수의 수 선택

AI는 Max Player이므로 가능한 후보 수 중 가장 높은 점수를 가진 수를 선택해야 한다. 이를 위해 처음에는 매우 작은 값으로 best_score를 설정하고, 각 후보 수의 점수가 더 높을 때마다 최적 점수와 최적 수를 갱신한다

```
def best_ai_move():
    best_score = -999
    best_move = None
    for move in legal_moves(board):
        board[move] = AI
        score = minimax(board, False)
        board[move] = EMPTY
        if score > best_score:
            best_score = score
            best_move = move
    return best_move
```

이 함수의 동작은 다음과 같이 정리할 수 있다.

- ① 현재 가능한 빈 칸을 모두 찾는다.
- ② 각 빈 칸에 AI가 O를 둔다고 가정한다.
- ③ Minimax로 해당 수의 점수를 계산한다.
- ④ 계산 후 보드를 원래 상태로 되돌린다.
- ⑤ 가장 높은 점수를 가진 칸을 best_move로 저장한다.
- ⑥ 최종적으로 best_move를 반환한다.

예를 들어 후보 수 A의 평가 점수가 3, 후보 수 B의 평가 점수가 7, 후보 수 C의 평가 점수가 5라고 하자. AI는 Max Player로서 가능한 후보 수들 가운데 가장 높은 평가값을 선택해야 하므로 세 점수에 대해 최대값 연산을 수행한다. 이때 $\max(3, 7, 5)=7$ 이 계산되며, 가장 큰 평가값 7을 가지는 후보 수 B가 최적의 선택으로 결정된다. 즉, AI는 각 후보 수의 미래 결과를 Minimax로 평가한 뒤, 그중 가장 높은 점수를 보장하는 수를 선택함으로써 자신의 이익을 최대화하게 된다.

최적 수 선택 과정 구현

지금까지는 Minimax 알고리즘이 가능한 수를 평가하고, 그중 가장 높은 점수를 가진 수를 선택하는 과정을 함수 단위로 살펴보았다. 다음 절에서는 지금까지 구현한 함수들을 하나의 프로그램으로 연결하여 VS Code에서 직접 실행하고, 실제 게임 진행 과정에서 AI가 어떤 방식으로 수를 선택하는지 확인한다

4. 전체 코드 실행 및 결과 분석

앞 절까지는 틱택토 게임 설계, 승패 판정, Minimax 재귀 함수, 최적 수 선택 과정을 각각 나누어 살펴보았다. 이번 절에서는 지금까지 작성한 기능들을 하나의 Python 파일로 연결하여 VS Code에서 직접 실행해 보고, 실제 게임 진행 과정에서 AI가 어떤 방식으로 수를 선택하는지 확인한다

(1) VS Code에서 실행하기

먼저 VS Code에서 새 Python 파일을 만든다.

tictactoe_minimax.py

이 파일 안에 아래의 전체 코드를 입력한다.

```
HUMAN = "X"
AI = "O"
EMPTY = " "
board = [EMPTY] * 9

def print_board():
    print()
    for i in range(3):
        row = board[i * 3:(i + 1) * 3]
        print(" | ".join(row))
        if i < 2:
            print("---+---+---")
    print()

def print_position_guide():
    print("칸 번호 안내")
    print("0 | 1 | 2")
    print("---+---+---")
    print("3 | 4 | 5")
    print("---+---+---")
    print("6 | 7 | 8")
    print()
```

최적 수 선택 과정 구현

```
def winner(b):
    win_cases = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8),
        (0, 3, 6), (1, 4, 7), (2, 5, 8),
        (0, 4, 8), (2, 4, 6)
    ]
    for a, c, d in win_cases:
        if b[a] == b[c] == b[d] and b[a] != EMPTY:
            return b[a]
    if EMPTY not in b:
        return "DRAW"
    return None

def legal_moves(b):
    return [i for i in range(9) if b[i] == EMPTY]

def minimax(b, is_maximizing):
    result = winner(b)

    if result == AI:
        return 10
    elif result == HUMAN:
        return -10
    elif result == "DRAW":
        return 0

    if is_maximizing:
        best_score = -999
        for move in legal_moves(b):
            b[move] = AI
            score = minimax(b, False)
            b[move] = EMPTY
            best_score = max(best_score, score)
        return best_score
    else:
        best_score = 999
        for move in legal_moves(b):
            b[move] = HUMAN
            score = minimax(b, True)
            b[move] = EMPTY
            best_score = min(best_score, score)
        return best_score

def best_ai_move():
    best_score = -999
    best_move = None
```

최적 수 선택 과정 구현

```
for move in legal_moves(board):
    board[move] = AI
    score = minimax(board, False)
    board[move] = EMPTY
    if score > best_score:
        best_score = score
        best_move = move
return best_move

def play_game():
    print("틱택토 Minimax 게임")
    print("플레이어: X / AI: O")
    print_position_guide()
    while True:
        print_board()
        # 사용자 입력
        try:
            move = int(input("X를 놓을 칸 번호를 입력하세요(0~8): "))
        except ValueError:
            print("숫자를 입력하세요.")
            continue
        if move not in range(9):
            print("0부터 8까지의 숫자만 입력하세요.")
            continue
        if board[move] != EMPTY:
            print("이미 말이 있는 칸입니다.")
            continue
        board[move] = HUMAN
        if winner(board):
            break
        # AI 차례
        ai_move = best_ai_move()
        board[ai_move] = AI
        print(f"AI가 {ai_move}번 칸에 O를 놓았습니다.")
        if winner(board):
            break
    print_board()
    result = winner(board)
    if result == HUMAN:
        print("플레이어 승리!")
    elif result == AI:
        print("AI 승리!")
    else:
        print("무승부입니다.")
    play_game()
```

최적 수 선택 과정 구현

VS Code 상단 메뉴에서 터미널을 실행한다.

Terminal → New Terminal

터미널에서 현재 파일이 저장된 폴더로 이동한 뒤 다음 명령어를 입력한다.

```
python tictactoe_minimax.py
```

정상적으로 실행되면 다음과 같이 틱택토 게임 안내와 칸 번호가 출력된다.

```
틱택토 Minimax 게임
플레이어: X / AI: O
칸 번호 안내
0 | 1 | 2
--+---+--
3 | 4 | 5
--+---+--
6 | 7 | 8
```

사용자는 0부터 8까지의 숫자 중 하나를 입력하여 원하는 칸에 x를 놓을 수 있다.

(2) 실행 결과 확인

프로그램이 실행되면 사용자가 먼저 x를 놓고, 그다음 AI가 Minimax 알고리즘을 이용해 o를 놓는다. 예를 들어 사용자가 4번 칸을 선택하면 가운데 칸에 x가 놓인다.

```

|   |
--+---+--
|   |
--+---+--
|   |
x를 놓을 칸 번호를 입력하세요(0~8): 4
```

이후 AI는 가능한 빈 칸을 모두 탐색한 뒤 가장 유리한 위치에 o를 둔다.

```
AI가 0번 칸에 O를 놓았습니다.
O |   |
--+---+--
| X |
--+---+--
|   |
```

사용자가 다시 빈 칸 번호를 입력하면, AI는 그 상태를 기준으로 다시 Minimax 탐색을 수행한다. 게임은 누군가 승리하거나 빈 칸이 모두 채워져 무승부가 될 때까지 반복된다. 최종적으로 프로그램은 게임 결과에 따라 AI 승리, 플레이어 승리, 또는 무승부입니다와 같은 메시지를 출력하며 게임을 종료한다.

5. 시간 복잡도 분석

Minimax 알고리즘을 이용해 가능한 수를 탐색하고 최적의 수를 선택하는 과정을 구현하였다. 그러나 이러한 방식은 모든 가능한 경우를 탐색한다는 점에서 계산량이 빠르게 증가하는 특징을 가진다. 따라서 Minimax 알고리즘을 이해하기 위해서는 탐색 과정에서 필요한 연산량, 즉 시간 복잡도를 함께 살펴볼 필요가 있다.

(1) 탐색 트리와 분기 인수

Minimax는 게임 상태를 트리(Tree) 구조로 확장하며 탐색한다. 현재 상태에서 가능한 수가 여러 개 존재하면 각각이 하나의 자식 노드가 되고, 다시 그 상태에서 또 여러 가능한 수가 생기면서 게임 트리가 형성된다.

이때 한 노드에서 생성되는 평균 가능한 수의 개수를 분기 인수(Branching Factor) 라고 하며 보통 b 로 나타낸다.

b = 한 상태에서 가능한 평균 수의 개수

또한 탐색하는 깊이를 d 라고 하면

d = 탐색 깊이

Minimax가 탐색해야 할 전체 노드 수는 대략 다음과 같이 표현된다.

$$1 + b + b^2 + b^3 + \dots + b^d$$

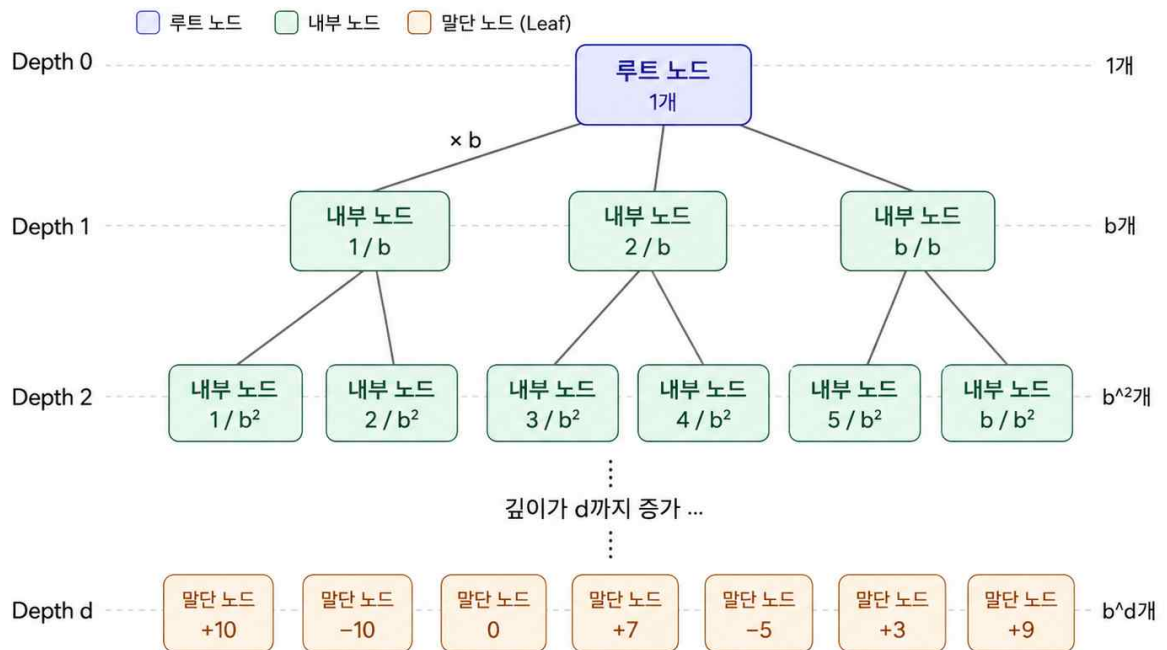
이는 근사적으로 $O(b^d)$ 의 시간 복잡도를 가진다.

(2) $O(b^d)$ 의 의미

탐색 깊이가 1 증가할 때마다 계산량이 b 배씩 증가한다는 뜻이다. 예를 들어 한 상태에서 가능한 수의 평균 개수, 즉 분기 인수가 5라고 가정하면 탐색 깊이가 증가할수록 노드 수는 지수적으로 증가한다. 깊이 1에서는 5개의 노드를 탐색하지만, 깊이 2에서는 $5^2 = 25$ 개, 깊이 3에서는 $5^3 = 125$ 개, 깊이 4에서는 $5^4 = 625$ 개의 노드를 탐색해야 한다. 이처럼 탐색 깊이가 한 단계 증가할 때마다 계산량이 5배씩 늘어나므로, Minimax의 탐색 비용은 매우 빠르게 커지게 된다.

이를 트리 형태로 보면 다음과 같다.

최적 수 선택 과정 구현



시간 복잡도: $O(b^d)$

b = 분기 인수(한 상태에서 가능한 평균 수의 개수), d = 탐색 깊이

즉 깊이가 조금만 증가해도 탐색해야 할 상태 수는 매우 빠르게 커진다.

(3) 틱택토에서의 시간 복잡도

틱택토는 3×3 보드이므로 최대 9개의 칸이 존재한다. 첫 수에서는 최대 9개, 다음은 8개, 이후 7개 ... 식으로 감소한다.

순수 탐색의 최대 경우의 수는 아래와 같이 362,880정도 이다.

$$9! = 9 \times 8 \times 7 \times \dots \times 1 = 362,880$$

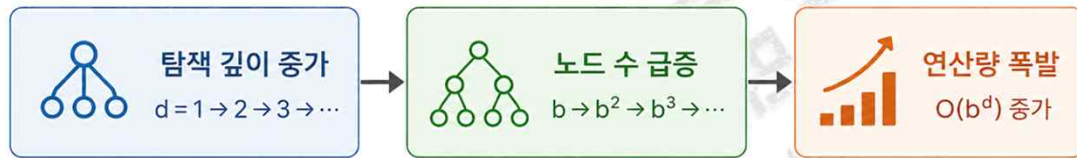
이는 컴퓨터가 충분히 계산 가능한 수준이므로 틱택토에서는 완전 탐색 기반 Minimax 구현이 가능하다. 하지만 체스는 상황이 다르다.

(4) 체스에서의 시간 복잡도

체스는 한 상태에서 가능한 수의 평균 개수, 즉 분기 인수가 약 $b \approx 30 \sim 35$ 정도로 알려져 있다. 따라서 탐색 깊이가 조금만 증가해도 계산량이 매우 빠르게 커진다. 예를 들어 깊이가 4까지만 탐색하더라도 $30^4 = 810,000$ 개의 상태를 검사해야 하며, 깊이가 6으로 증가하면 $30^6 = 729,000,000$ 개의 상태를 탐색해야 한다. 이처럼 체스와 같이 분기 인수가 큰 게임에서는 탐색 깊이가 증가할수록 계산량이 폭발적으로 증가하며, 이것이 Minimax 알고리즘에서 계산 비용 문제가 발생하는 주요 원인이다.

최적 수 선택 과정 구현

이는 매우 큰 계산량이며, 이를 탐색 공간 폭발(Search Space Explosion) 이라고 한다.



이 문제가 Minimax의 대표적 한계이다.

(5) 시간 복잡도와 성능 한계

Minimax는 최적 수를 보장할 수 있다는 장점이 있지만, 탐색 깊이가 커질수록 계산 비용이 매우 커진다.

✓ 장점	✗ 단점
 최적 해 보장 모든 경우 탐색 → 항상 최선의 수	 계산량 급증 $O(b^d)$ — 깊이마다 b배 증가
 상대 대응까지 고려 Min 노드로 상대 전략 반영	 깊은 탐색에서 비효율 체스·바둑은 단독 사용 불가
 결과의 신뢰도 높음 완전한 탐색 기반이므로 예측 가능	 메모리 사용 증가 탐색 트리 저장으로 메모리 부담

표에서 정리한 것처럼 Minimax 알고리즘은 모든 가능한 경우를 탐색하여 최적 해를 보장하고, 상대의 대응까지 고려할 수 있다는 장점을 가진다. 이러한 특성 덕분에 게임 AI의 기본 원리를 이해하고 최적 의사결정을 구현하는 데 매우 적합하다. 특히 완전 탐색 기반 접근이므로 계산된 결과의 신뢰도가 높다는 점도 중요한 장점이다. 반면 Minimax는 탐색 깊이가 증가할수록 계산량이 급격히 증가하는 한계를 가진다. 시간 복잡도가 $O(b^d)$ 형태로 증가하므로 깊은 탐색에서는 연산량 부담이 매우 커지며, 체스나 바둑과 같이 분기 인수가 큰 게임에서는 순수 Minimax만으로 효율적인 탐색이 어렵다. 또한 탐색 트리 확장에 따른 메모리 사용 증가 역시 성능 저하 요인이 된다.

(6) 최적화 필요성

이와 같은 계산량 문제 때문에 실제 게임 인공지능에서는 순수 Minimax 알고리즘만 단독으로 사용하지 않는다. Minimax는 모든 가능한 수를 탐색한다는 점에서 최적 해를 보장하지만, 탐색 깊이가 증가할수록 연산량이 급격히 증가하므로 복잡한 게임에서는 비효율적일 수 있다. 따라서 탐색 과정에서 결과에 영향을 주지 않는 불필요한 분기를 제거하여 계산량을 줄이는 최적화 기법이 필요하다.

대표적인 방법이 다음 장에서 다룰 Alpha-Beta Pruning(알파-베타 가지치기)이다. 이 기법은 탐색 도중 더 이상 살펴볼 필요가 없는 가지(branch)를 미리 제거함으로써, 최종의 사결정 결과는 유지하면서도 탐색해야 할 노드 수를 크게 줄일 수 있다. 즉, 최적 수를 찾는 정확성은 유지하면서 계산 효율을 향상시키는 방법이다.

기본 Minimax의 시간 복잡도는 $O(b^d)$ 로 표현되지만, Alpha-Beta Pruning을 적용하면 이론적으로 $O(b^{d/2})$ 수준까지 감소할 수 있다. 이는 탐색 깊이가 동일하더라도 실제 계산량을 크게 줄일 수 있음을 의미한다. 따라서 Alpha-Beta Pruning은 Minimax의 계산량 한계를 보완하고, 실제 체스와 같은 복잡한 게임에서 탐색 기반 AI를 현실적으로 구현할 수 있게 해 주는 핵심 최적화 기법이라 할 수 있다. 다음 절에서는 Alpha와 Beta 값이 어떻게 불필요한 탐색을 제거하여 Minimax 성능을 향상시키는지 살펴본다.